C3 AI Coding — 24-Page Hit List

Contents

C3 AI Coding — 24-Page Hit List

Use this as a speak-code-test drill: name the invariant, trace the tiny state, write the optimal version from memory, then explain the alternate. Mark misses and repeat only those; the goal is fast pattern recognition, not passive reading.

Dense pattern cheat sheet

Signal	Default pattern	Invariant / question	Cost
exact pair / frequency	set or `Counter`	have I seen the complement?	O(n)
contiguous sum = k	prefix sum + counts	prior prefix `p-k` completes range	O(n)
monotone feasibility	binary search answer	first feasible or last feasible?	O(n log R)
sorted rotated data	modified binary search	which half is sorted?	O(log n)
minimum combinations	bottom-up DP	best answer for every smaller amount	O(nA)
grid shortest waves	multi-source BFS	queue is current frontier	O(rc)
enumerate choices	backtracking	choose, recurse, undo	exponential
overlapping intervals	sort then sweep	output is disjoint and sorted	O(n log n)
matching delimiters	stack	top is only legal closer target	O(n)
water / opposing bounds	two pointers	smaller boundary decides safely	O(n)
next item by recency	hash map + DLL	MRU and LRU updates are O(1)	O(1)
circular tour	greedy reset	failed prefix cannot be a start	O(n)
linked-list offset	fast/slow pointers	fixed gap equals n	O(n)
top k ranked values	heap or bucket	heap retains only best k	O(n log k)
canonical path	stack	stack is normalized path	O(n)

Fast interview script: clarify empty/negative/duplicate constraints → state brute force → name invariant → trace one boundary case → code → test → complexity. Prefer integer arithmetic; seed prefix counts with `{0:1}`; use a dummy linked-list head; in BFS mark visited when enqueued; in backtracking always undo.

1. Fraction to Recurring Decimal (166)

Method: Long division; map each remainder to its first output position. - `1/6`: integer `0`, remainder `1`. - emit `1`; remainder `4`; remember `1 → 2`. - emit `6`; remainder `4` repeats: insert `(` at saved position. - result

`0.1(6)`.

```
def fractionToDecimal(n, d):
    if n == 0: return "0"
    sign = "-" if (n < 0) ^ (d < 0) else ""
    n, d = abs(n), abs(d)
    q, r = divmod(n, d)
    if not r: return sign + str(q)
    out, seen = list(sign + str(q) + "."), {}
    while r and r not in seen:
        seen[r] = len(out)
        q, r = divmod(r * 10, d)
        out.append(str(q))
    if r:
        out.insert(seen[r], "("); out.append(")")
    return "".join(out)
```

Alternate trick: Denominator factors only of 2 and 5 terminate after reduction; otherwise a repeat is guaranteed. Never use float—precision cannot identify the cycle.

2. K-diff Pairs in an Array (532)

Method: Count values; for `k>0` count `x+k`, for `k=0` count duplicates. - `[3,1,4,1,5]`, `k=2` → keys `{1,3,4,5}`. - `1→3` yes; `3→5` yes; others no. - Answer `2`; duplicate `1` still contributes one distinct pair.

```
from collections import Counter
def findPairs(nums, k):
    if k < 0: return 0
    c = Counter(nums)
    if k == 0:
        return sum(v > 1 for v in c.values())
    return sum(x + k in c for x in c)
```

Alternate trick: Sort unique values and use two pointers for O(n log n) time, O(1) extra after sort; useful when mutation is allowed.

3. Koko Eating Bananas (875)

Method: Binary-search the minimum speed whose required hours are at most `h`. - `[3,6,7,11]`, `h=8`; search speeds `1..11`. - `k=6` → hours `1+1+2+2=6`, feasible; move left. - `k=3` → `1+2+3+4=10`, infeasible; move right. - first feasible speed is `4`.

```
def minEatingSpeed(piles, h):
    lo, hi = 1, max(piles)
    while lo < hi:
        mid = (lo + hi) // 2
        hours = sum((p + mid - 1) // mid for p in piles)
        if hours <= h: hi = mid
        else: lo = mid + 1
    return lo
```

Alternate trick: `ceil(p/k)` is `(p+k-1)//k`; avoid floats. State the monotonic predicate: increasing speed never increases hours.

4. Unique Paths (62)

Method: 1-D DP; each cell becomes `from_left + from_above`. - For `3×2`, start row `[1,1,1]`. - Second row updates `[1,2,3]`. - Destination is `3`.

```python
def uniquePaths(m, n):
    dp = [1] * n
    for _ in range(1, m):
        for c in range(1, n):
            dp[c] += dp[c - 1]
    return dp[-1]
```

Alternate trick: Choose the down moves among all moves: `C(m+n-2, m-1)`.

```python
from math import comb
# return comb(m + n - 2, m - 1)
```

5. Coin Change (322)

Method: Bottom-up DP where `dp[a]` is the fewest coins totaling `a`. - Coins `[1,2,5]`; `dp[0]=0`, all others infinity. - `dp[1]=1`, `dp[2]=1`, `dp[5]=1`. - `dp[10]=2`; `dp[11]=dp[10]+1=3`.

```python
def coinChange(coins, amount):
    dp = [amount + 1] * (amount + 1)
    dp[0] = 0
    for a in range(1, amount + 1):
        for c in coins:
            if c <= a:
                dp[a] = min(dp[a], dp[a - c] + 1)
    return -1 if dp[amount] > amount else dp[amount]
```

Alternate trick: BFS over reachable totals finds the first minimum-depth solution, but DP is simpler and has the same O(amount × coins) bound.

6. Valid Parentheses (20)

Method: Push expected closing symbols; each closer must equal the stack top. - `{[]}`: push `}`, `]`, `)` as openings arrive. - each closer pops its exact expected value. - `([)]` fails when `)` arrives but top expects `]`.

```python
def isValid(s):
    expect, st = {"(": ")", "[": "]", "{": "}"}, []
    for ch in s:
        if ch in expect: st.append(expect[ch])
        elif not st or st.pop() != ch: return False
    return not st
```

Alternate trick: Repeatedly remove `()`, `[]`, `{}` until unchanged; concise but $O(n^2)$, so mention only as brute force.

7. Trapping Rain Water (42)

Method: Two pointers; advance the side with the smaller running maximum. - At every step, trapped water is `boundary - height`. - If `left_max <= right_max`, future right heights cannot limit the left. - For `[4,2,0,3,2,5]`, left contributions are `2,4,1,2`.

```python
def trap(h):
    l, r, lm, rm, water = 0, len(h)-1, 0, 0, 0
    while l <= r:
        if lm <= rm:
            lm = max(lm, h[l]); water += lm - h[l]; l += 1
        else:
            rm = max(rm, h[r]); water += rm - h[r]; r -= 1
```

```
    return water
```

Alternate trick: A decreasing stack computes bounded basins in O(n), but two pointers use O(1) space and are easier to prove.

8. Rotting Oranges (994)

Method: Multi-source BFS from every rotten orange; count fresh oranges. - enqueue all `2`s at minute `0`; fresh count starts at number of `1`s. - each layer rots adjacent fresh cells and decrements `fresh`. - answer elapsed layers, or `-1` if fresh remains.

```
from collections import deque
def orangesRotting(g):
    q = deque(); fresh = 0
    for r, row in enumerate(g):
        for c, x in enumerate(row):
            if x == 2: q.append((r, c, 0))
            elif x == 1: fresh += 1
    minutes = 0
    while q:
        r, c, minutes = q.popleft()
        for dr, dc in ((1,0),(-1,0),(0,1),(0,-1)):
            nr, nc = r+dr, c+dc
            if 0 <= nr < len(g) and 0 <= nc < len(g[0]) and g[nr][nc] == 1:
                g[nr][nc] = 2; fresh -= 1; q.append((nr,nc,minutes+1))
    return minutes if fresh == 0 else -1
```

Alternate trick: Process `len(q)` nodes per level instead of storing time; increment minutes only when another frontier exists.

9. Merge Intervals (56)

Method: Sort by start; merge into the last emitted interval. - Sorted: `[1,3],[2,6],[8,10],[15,18]`. - `[2,6]` overlaps last end `3`, extend end to `6`. - gaps start new output intervals.

```
def merge(intervals):
    intervals.sort()
    out = []
    for s, e in intervals:
        if not out or s > out[-1][1]:
            out.append([s, e])
        else:
            out[-1][1] = max(out[-1][1], e)
    return out
```

Alternate trick: Event-line sweeps generalize to overlap counts, but sorting and merging is optimal and clearer for union output.

10. LRU Cache (146)

Method: Hash map plus ordered dictionary; reads and writes move keys to MRU. - capacity `2`: put `1`, put `2` → order `[1,2]`. - get `1` → `[2,1]`; put `3` evicts `2`. - left is LRU; right is MRU.

```
from collections import OrderedDict
class LRUCache:
    def __init__(self, capacity):
        self.cap, self.d = capacity, OrderedDict()
```

```
    def get(self, key):
        if key not in self.d: return -1
        self.d.move_to_end(key)
        return self.d[key]
    def put(self, key, value):
        if key in self.d: self.d.move_to_end(key)
        self.d[key] = value
        if len(self.d) > self.cap: self.d.popitem(last=False)
```

Alternate trick: Implement a sentinel-head/tail doubly linked list plus map when library ordering is disallowed; all four link edits remain O(1).

11. Subarray Sum Equals K (560)

Method: Prefix-sum frequency map; count prior prefixes equal to `prefix-k`. - `[1,1,1]`, `k=2`; seed `{0:1}`. - prefixes `1,2,3`; needed values `-1,0,1`. - matches at prefixes `2` and `3`; answer `2`.

```
from collections import defaultdict
def subarraySum(nums, k):
    freq = defaultdict(int); freq[0] = 1
    pref = ans = 0
    for x in nums:
        pref += x
        ans += freq[pref - k]
        freq[pref] += 1
    return ans
```

Alternate trick: Sliding window fails with negatives because expanding is not monotone. It works only under nonnegative constraints.

12. Generate Parentheses (22)

Method: Backtrack, adding `(` if available and `)` only when it cannot exceed opens. - `n=2`: start `("", open=0, close=0)`. - choose `(` → either `(` then `))`, or `)` then `()`. - leaves: `(())`, `()()`.

```
def generateParenthesis(n):
    out = []
    def dfs(s, op, cl):
        if len(s) == 2*n: out.append(s); return
        if op < n: dfs(s+"(", op+1, cl)
        if cl < op: dfs(s+")", op, cl+1)
    dfs("", 0, 0)
    return out
```

Alternate trick: Generate all `2^(2n)` strings then validate is wasteful; pruning produces only valid prefixes, O(Catalan(n) × n).

Top-12 deeper walkthrough drills

Use these as verbal traces. Before reading the last column, say the invariant and the next mutation. The tables are intentionally small enough to reproduce on a whiteboard.

1 — Fraction to Recurring Decimal: `4/333` | Step | Emitted | Remainder before ×10 | Digit / new remainder | Map action | |—|—|—:|—|—| | integer | `0.` | 4 | — | save `4 → 2` | | 1 | `0.0` | 4 | `0 / 40` | save `40 → 3` | | 2 | `0.01` | 40 | `1 / 67` | save `67 → 4` | | 3 | `0.012` | 67 | `2 / 4` | 4 repeats at index 2 | | finish | `0.(012)` | — | — | insert parentheses, do not divide again |

The remainder, not the digit, identifies the complete future state. At most `|d|` distinct remainders occur,

so the loop terminates or repeats.

2 — K-diff Pairs: `[1,1,1,2,2]` | Case | Keys / counts | Test | Contribution | |—|—|—|—:| | `k=0` | `1:3, 2:2` | count greater than one | `2` | | `k=1` | keys `{1,2}` | test `x+1` once per key | `1` | | `k<0` | absolute differences cannot be negative | reject | `0` | | duplicates | many index pairs exist | problem asks distinct value pairs | no extra |

Clarify whether the interviewer means distinct values or distinct index pairs; the implementation changes completely.

3 — Koko: first-true binary search | `lo` | `hi` | `mid` | Hours for `[3,6,7,11]` | Decision | |—:|—:|—:|—:|—| | 1 | 11 | 6 | 6 | feasible, keep 6 and left half | | 1 | 6 | 3 | 10 | infeasible, discard through 3 | | 4 | 6 | 5 | 8 | feasible, keep 5 and left half | | 4 | 5 | 4 | 8 | feasible; converge to 4 |

Loop contract: every speed below `lo` is known infeasible; `hi` remains feasible. This explains `while lo < hi` and `hi = mid`.

4 — Unique Paths: rolling row for `3×4` | Row processed | DP state | Interpretation | |—:|—|—| | 0 | `[1,1,1,1]` | only right moves along top | | 1 col 1 | `[1,2,1,1]` | above 1 + left 1 | | 1 complete | `[1,2,3,4]` | paths to second row | | 2 complete | `[1,3,6,10]` | destination has 10 paths |

Iteration direction matters: left-to-right keeps `dp[c]` as old "above" while `dp[c-1]` is new "left."

5 — Coin Change: `[1,3,4]`, amount `6` | Amount | Best predecessor | `dp[a]` | Why | |—:|—|—:|—| | 0 | base | 0 | no coins | | 1 | `dp[0]+1` | 1 | coin 1 | | 2 | `dp[1]+1` | 2 | two 1s | | 3 | `dp[0]+1` | 1 | coin 3 | | 4 | `dp[0]+1` | 1 | coin 4 | | 5 | `dp[4]+1` | 2 | 4 + 1 | | 6 | `dp[3]+1` | 2 | 3 + 3, not greedy 4 + 1 + 1 |

The sentinel `amount+1` is safe because no valid answer needs more than `amount` positive coins when coin 1 exists.

6 — Valid Parentheses: `([{}])` | Input | Expected-close stack | Action | |—|—|—| | `(` | `)` | push expected closer | | `[` | `),]` | push | | `{` | `),],}` | push | | `}` | `),]` | matches and pops | | `]` | `)` | matches and pops | | `)` | empty | matches; final empty means valid |

Storing expected closers removes a second lookup on close and makes the mismatch test one expression.

7 — Trapping Water: boundary proof | State | Smaller known max | Safe side | Added water | |—|—|—|—:| | `lm=4, rm=5`, left height 2 | left max | left | 2 | | `lm=4, rm=5`, left height 0 | left max | left | 4 | | `lm=4, rm=5`, left height 3 | left max | left | 1 | | equal maxima | either | choose consistently | boundary minus height |

Once `lm <= rm`, some right wall at least `lm` already exists. Unknown interior/right values cannot reduce water at the current left cell.

8 — Rotting Oranges: frontier timing | Minute | Queue frontier | Fresh changed | Remaining | |—:|—|—|—:| | 0 | all initially rotten cells | none | initial count | | 1 | neighbors reached from minute 0 | mark on enqueue | decreases once per cell | | 2 | next unvisited neighbors | mark on enqueue | decreases | | end | queue empty | none | zero means success |

Marking on enqueue prevents two rotten parents from enqueueing and decrementing the same fresh orange twice.

9 — Merge Intervals: touching policy | Last output | Candidate | Condition | Result | |—|—|—|—| | `[1,3]` | `[2,6]` | `2 <= 3` | extend to `[1,6]` | | `[1,6]` | `[8,10]` | `8 > 6` | append | | `[8,10]` | `[10,12]` | `10 <= 10` | merge if endpoints are closed | | `[8,12]` | `[9,11]` | contained | unchanged |

Ask whether ranges are closed, half-open, or timestamps; "touching" intervals may or may not overlap.

10 — LRU Cache: pointer-level trace | Operation | LRU → MRU | Map mutation | Eviction | |—|—|—|—| | `put(1,A)` | `1` | add node 1 | none | | `put(2,B)` | `1,2` | add node 2 | none | | `get(1)` | `2,1` | same node moved | none | | `put(3,C)` | `1,3` | add 3, remove 2 | key 2 | | `put(1,Z)` | `3,1` | update and move 1 | none |

With sentinels, remove is always four assignments and append is always four assignments; no endpoint branches are needed.

11 — Subarray Sum Equals K: `[3,4,7,2,-3,1,4,2]`, `k=7` | Index / value | Prefix | Need `prefix-k` | Prior count added | |—|—:|—:|—:| | seed | 0 | — | — | | 0 / 3 | 3 | -4 | 0 | | 1 / 4 | 7 | 0 | 1 | | 2 / 7 | 14 | 7 | 1 | | 4 / -3 | 13 | 6 | 0 | | 5 / 1 | 14 | 7 | 1 | | 7 / 2 | 20 | 13 | 1 |

Query the old frequency before incrementing the current prefix. This counts nonempty ranges and naturally handles duplicate prefix values.

12 — Generate Parentheses: `n=3` state pruning | Prefix | `(open,close)` | Legal next choices | Reason | |—|—|—|—| | empty | `(0,0)` | `(` | cannot close empty prefix | | `((` | `(2,0)` | `(` or `)` | opens remain; close is balanced-safe | | `()` | `(1,1)` | `(` | close would exceed opens | | `(((` | `(3,0)` | `)` | open budget exhausted | | `(()` | `(2,1)` | `(` or `)` | both guards pass |

Every recursive state is a valid prefix. Leaves therefore need no validator, and the output-size lower bound dominates runtime.

13. Container With Most Water (11)

Method: Two pointers; compute area, then discard the shorter wall. - Area is `(r-l) × min(h[l],h[r])`. - Moving the taller wall cannot improve the limiting height. - `[1,8,6,2,5,4,8,3,7]`: best is indices `1,8`, area `49`.

```
def maxArea(h):
    l, r, best = 0, len(h)-1, 0
    while l < r:
        best = max(best, (r-l) * min(h[l], h[r]))
        if h[l] <= h[r]: l += 1
        else: r -= 1
    return best
```

Alternate trick: O(n^2) checks every pair; use it only to explain why discarding the shorter side eliminates dominated pairs.

14. Spiral Matrix (54)

Method: Shrink top, right, bottom, left boundaries after traversing each edge. - `3×3`: top row `1,2,3`; right side `6,9`. - bottom reversed `8,7`; left upward `4`. - center `5`; guard bottom and left passes after shrinking.

```
def spiralOrder(a):
    out, top, bot, left, right = [], 0, len(a)-1, 0, len(a[0])-1
    while top <= bot and left <= right:
        out += a[top][left:right+1]; top += 1
        for r in range(top, bot+1): out.append(a[r][right])
        right -= 1
        if top <= bot: out += a[bot][left:right+1][::-1]; bot -= 1
        if left <= right:
            for r in range(bot, top-1, -1): out.append(a[r][left])
            left += 1
    return out
```

Alternate trick: Direction simulation with a visited set is O(mn) space; boundary shrinking is O(1) auxiliary space.

15. Gas Station (134)

Method: If total gas covers total cost, greedily reset start after any negative tank. - Gains for each station are `gas[i]-cost[i]`. - If tank becomes negative at `i`, no start since last reset can pass `i`. - reset candidate to `i+1`; global total decides existence.

```python
def canCompleteCircuit(gas, cost):
    total = tank = start = 0
    for i, (g, c) in enumerate(zip(gas, cost)):
        gain = g - c; total += gain; tank += gain
        if tank < 0:
            start, tank = i + 1, 0
    return start if total >= 0 else -1
```

Alternate trick: Duplicated-array prefix minima can derive the same start, but greedy reset is simpler and O(1) space.

16. Group the People (1282)

Method: Bucket indices by required size; emit a bucket immediately when full. - Sizes `[3,3,3,3,3,1,3]`. - size-3 bucket emits `[0,1,2]`, then resets. - size-1 emits `[5]`; remaining size-3 emits `[3,4,6]`.

```python
from collections import defaultdict
def groupThePeople(sizes):
    buckets, out = defaultdict(list), []
    for person, size in enumerate(sizes):
        buckets[size].append(person)
        if len(buckets[size]) == size:
            out.append(buckets[size])
            buckets[size] = []
    return out
```

Alternate trick: Sort `(size,index)` and slice runs into groups; O(n log n), useful when deterministic ordering is requested.

17. Top K Frequent Words (692)

Method: Count, then sort words by descending frequency and ascending lexicographic order. - `["i","love","leetcode","i","love","coding"]`, `k=2`. - counts: `i=2`, `love=2`, others `1`. - tie breaks alphabetically → `["i","love"]`.

```python
from collections import Counter
def topKFrequent(words, k):
    c = Counter(words)
    return sorted(c, key=lambda w: (-c[w], w))[:k]
```

Alternate trick: A size-k heap gives O(u log k), but Python tie direction needs a custom wrapper; full sorting is safer unless `u` is huge.

18. Search in Rotated Sorted Array (33)

Method: Modified binary search; identify the sorted half, then keep the half containing target. - `[4,5,6,7,0,1,2]`, target `0`; mid `7`. - left half sorted, target not inside → move right. - mid `1`; left half `[0,1]` contains target → move left.

```python
def search(nums, target):
    l, r = 0, len(nums)-1
    while l <= r:
        m = (l+r)//2
        if nums[m] == target: return m
        if nums[l] <= nums[m]:
            if nums[l] <= target < nums[m]: r = m-1
            else: l = m+1
```

```
        else:
            if nums[m] < target <= nums[r]: l = m+1
            else: r = m-1
    return -1
```

Alternate trick: Find pivot then ordinary binary search; same O(log n), but two phases add code. Duplicates require shrinking ambiguous equal ends.

19. Word Search (79)

Method: DFS from matching cells; mark a cell during the path, then restore it. - Start only where board cell equals `word[0]`. - state is `(row,col,next_index)`; four choices per step. - a cell may be reused by another path, never the current path.

```
def exist(b, word):
    R, C = len(b), len(b[0])
    def dfs(r, c, i):
        if i == len(word): return True
        if not (0 <= r < R and 0 <= c < C) or b[r][c] != word[i]: return False
        ch, b[r][c] = b[r][c], "\0"
        ok = any(dfs(r+dr,c+dc,i+1) for dr,dc in ((1,0),(-1,0),(0,1),(0,-1)))
        b[r][c] = ch
        return ok
    return any(dfs(r,c,0) for r in range(R) for c in range(C))
```

Alternate trick: Precheck board/word character counts and reverse the word if its last character is rarer; often cuts branching without changing worst-case O(RC · 4^L).

20. Design Hit Counter (362)

Method: Queue `(timestamp,count)` buckets; evict entries at least 300 seconds old. - hits at `1,2,300,300` store `(1,1),(2,1),(300,2)`. - query `300` keeps all four. - query `301` evicts timestamp `1`; total becomes `3`.

```
from collections import deque
class HitCounter:
    def __init__(self): self.q, self.total = deque(), 0
    def hit(self, t):
        if self.q and self.q[-1][0] == t: self.q[-1][1] += 1
        else: self.q.append([t, 1])
        self.total += 1
    def getHits(self, t):
        while self.q and self.q[0][0] <= t - 300:
            self.total -= self.q.popleft()[1]
        return self.total
```

Alternate trick: Fixed 300-slot arrays store timestamp and count; each query scans 300 slots—strict O(1) with a small constant and bounded memory.

21. Continuous Subarray Sum (523)

Method: Store the earliest index of each prefix remainder; equal remainders imply a divisible subarray. - `[23,2,4,6,7]`, `k=6`; seed remainder `0` at index `-1`. - remainders: `5` at 0, `1` at 1, `5` at 2. - repeated `5`, distance `2` → `[2,4]` sums to `6`.

```
def checkSubarraySum(nums, k):
    first, pref = {0: -1}, 0
    for i, x in enumerate(nums):
```

```
        pref += x
        rem = pref % k if k else pref
        if rem in first:
            if i - first[rem] >= 2: return True
        else:
            first[rem] = i
    return False
```

Alternate trick: Keep the earliest index only—overwriting it can destroy the required length-two gap. For `k=0`, repeated raw prefix sums handle zero-sum ranges.

22. Two City Scheduling (1029)

Method: Sort by relative cost `A-B`; send first half to A, rest to B. - `[10,20]` difference `-10`: strong A preference. - `[400,50]` difference `350`: strong B preference. - sorting chooses globally cheapest opportunity costs.

```
def twoCitySchedCost(costs):
    costs.sort(key=lambda x: x[0] - x[1])
    n = len(costs) // 2
    return sum(a for a, _ in costs[:n]) + sum(b for _, b in costs[n:])
```

Alternate trick: Start with everyone in A, then add the `n` smallest switching costs `B-A`; same proof, sometimes easier to explain.

23. Simplify Path (71)

Method: Split on `/`; stack normal names, ignore empty/dot, pop on double-dot. - `/a/./b/../../c/` tokens normalize incrementally. - `a` push; `.` ignore; `b` push; two `..` pop. - `c` remains → `/c`.

```
def simplifyPath(path):
    st = []
    for part in path.split("/"):
        if part in ("", "."): continue
        if part == "..":
            if st: st.pop()
        else: st.append(part)
    return "/" + "/".join(st)
```

Alternate trick: Regex normalization is brittle around root and repeated separators; the stack directly models directory semantics in O(n).

24. Remove Nth Node From End of List (19)

Method: Dummy head; advance fast `n` steps, then move both until fast is last. - `1→2→3→4→5`, `n=2`; dummy avoids deleting-head special case. - fixed gap of two puts slow before node `4`. - bypass `slow.next`; result `1→2→3→5`.

```
def removeNthFromEnd(head, n):
    dummy = ListNode(0, head)
    fast = slow = dummy
    for _ in range(n): fast = fast.next
    while fast.next:
        fast, slow = fast.next, slow.next
    slow.next = slow.next.next
    return dummy.next
```

Alternate trick: First compute length, then delete position `length-n`; two passes are still O(n), but the

one-pass gap is the expected interview solution.

25. Time Based Key-Value Store (981)

Method: Keep append-ordered `(timestamp,value)` lists per key; binary-search the last timestamp not exceeding the query. - `set("model","v1",2)`, `set("model","v2",8)`. - `get("model",6)` chooses the rightmost timestamp `<=6`, returning `v1`. - `get("model",1)` has no valid predecessor and returns empty.

```
from collections import defaultdict
from bisect import bisect_right
class TimeMap:
    def __init__(self):
        self.d = defaultdict(list)
    def set(self, key, value, timestamp):
        self.d[key].append((timestamp, value))
    def get(self, key, timestamp):
        a = self.d[key]
        i = bisect_right(a, (timestamp, chr(0x10ffff))) - 1
        return a[i][1] if i >= 0 else ""
```

Invariant: Lists are timestamp-sorted because writes arrive in increasing order. If that guarantee disappears, use an ordered map or sort before serving. **Cost:** O(1) append; O(log n) lookup per key; O(total writes) space. **C3 angle:** Versioned configuration and model metadata often need "value effective as of event time."

26. Task Scheduler (621)

Method: The most frequent task creates frames; idle slots depend on maximum frequency and how many tasks share it. - Counts `A:3,B:3`, cooldown `2`. - Skeleton `A _ _ A _ _ A`; place B into slots to get `A B _ A B _ A B`. - Formula lower bound is `(max_freq-1)*(n+1)+num_max`.

```
from collections import Counter
def leastInterval(tasks, n):
    c = Counter(tasks)
    mx = max(c.values())
    tied = sum(v == mx for v in c.values())
    return max(len(tasks), (mx - 1) * (n + 1) + tied)
```

Invariant: All but the final group after a maximum-frequency task require `n` separating positions. **Alternate:** A max heap plus cooldown queue simulates execution and generalizes to returning an actual schedule. **Cost:** O(n) counting with bounded alphabet; heap simulation O(T log u).

27. Meeting Rooms II (253)

Method: Sort starts and ends independently; sweep the next chronological event. - Starts `[0,5,15]`, ends `[10,20,30]`. - Start 0 → room 1; start 5 before end 10 → room 2. - End 10 before start 15 releases one; peak remains 2.

```
def minMeetingRooms(intervals):
    starts = sorted(s for s, _ in intervals)
    ends = sorted(e for _, e in intervals)
    i = j = rooms = best = 0
    while i < len(starts):
        if starts[i] < ends[j]:
            rooms += 1; best = max(best, rooms); i += 1
        else:
            rooms -= 1; j += 1
```

```
    return best
```

Boundary: With half-open meetings `[start,end)`, an ending at 10 frees a room for a start at 10, hence strict `<`. **Alternate:** Sort intervals and keep active end times in a min heap, O(n log n), and retain room assignments.

28. Number of Islands (200)

Method: Scan cells; each unseen land cell starts a DFS/BFS that erases one entire component. - The scan discovers an island exactly once at its first remaining `1`. - Mark before exploring neighbors to avoid cycles. - Mutation is visited storage; use a set if input must remain unchanged.

```
def numIslands(g):
    if not g: return 0
    R, C, ans = len(g), len(g[0]), 0
    def flood(r, c):
        if not (0 <= r < R and 0 <= c < C) or g[r][c] != "1": return
        g[r][c] = "0"
        for dr, dc in ((1,0),(-1,0),(0,1),(0,-1)):
            flood(r+dr, c+dc)
    for r in range(R):
        for c in range(C):
            if g[r][c] == "1":
                ans += 1; flood(r, c)
    return ans
```

Cost: O(RC) time, O(RC) worst-case recursion stack. Iterative BFS avoids recursion-depth failure. **C3 angle:** Connected regions, dependency clusters, and topology grouping are recurring data-shape questions.

29. Clone Graph (133)

Method: DFS/BFS with a map from original node identity to clone identity. - Create the clone before recursing into neighbors. - A cycle then finds the existing clone instead of recursing forever. - Append cloned neighbors in original adjacency order.

```
def cloneGraph(node):
    copies = {}
    def dfs(x):
        if not x: return None
        if x in copies: return copies[x]
        copies[x] = Node(x.val)
        copies[x].neighbors = [dfs(y) for y in x.neighbors]
        return copies[x]
    return dfs(node)
```

Invariant: Every discovered original has exactly one allocated clone, even if its adjacency list is incomplete. **Wrong approach:** Mapping by `node.val` fails when values are not unique; object identity is the key. **Cost:** O(V+E) time and O(V) auxiliary space.

30. Course Schedule (207)

Method: Topological sort; repeatedly consume zero-indegree courses. - Edge `prerequisite → course`; indegree counts unmet prerequisites. - Every consumed node releases its outgoing neighbors. - If fewer than `n` nodes are consumed, a directed cycle remains.

```
from collections import deque
def canFinish(n, prerequisites):
```

```
    g, indeg = [[] for _ in range(n)], [0] * n
    for course, pre in prerequisites:
        g[pre].append(course); indeg[course] += 1
    q = deque(i for i, d in enumerate(indeg) if d == 0)
    done = 0
    while q:
        x = q.popleft(); done += 1
        for y in g[x]:
            indeg[y] -= 1
            if indeg[y] == 0: q.append(y)
    return done == n
```

Alternate: Three-color DFS detects a back edge. Kahn's algorithm naturally returns an execution order. **C3 angle:** Pipeline dependencies, feature computation DAGs, and job orchestration map directly to topological ordering.

31. Lowest Common Ancestor of a Binary Tree (236)

Method: Postorder recursion; return a found target upward, or current node when targets arrive from both sides. - If current is `p` or `q`, return it. - If left and right both return non-null, current is the split point. - Otherwise propagate whichever side found a target.

```
def lowestCommonAncestor(root, p, q):
    if not root or root is p or root is q: return root
    left = lowestCommonAncestor(root.left, p, q)
    right = lowestCommonAncestor(root.right, p, q)
    if left and right: return root
    return left or right
```

Assumption: Both targets exist. If not guaranteed, return presence flags so one found node is not mistaken for a valid LCA. **Cost:** O(n) time, O(h) call stack. **Talk track:** "Each subtree reports evidence; the first node receiving evidence from both branches is the answer."

32. Binary Tree Level Order Traversal (102)

Method: BFS; snapshot queue length before processing each level. - Nodes already in the queue form exactly the current depth. - Children appended during the loop belong to the next depth. - Collect one list per snapshot.

```
from collections import deque
def levelOrder(root):
    if not root: return []
    q, out = deque([root]), []
    while q:
        level = []
        for _ in range(len(q)):
            x = q.popleft(); level.append(x.val)
            if x.left: q.append(x.left)
            if x.right: q.append(x.right)
        out.append(level)
    return out
```

Cost: O(n) time, O(width) queue space. **Extension:** Right-side view takes the last node per level; zigzag reverses alternate output levels without changing traversal.

33. Product of Array Except Self (238)

Method: Write prefix products into output, then multiply by a rolling suffix product. - `[1,2,3,4]` prefix output becomes `[1,1,2,6]`. - Rolling suffixes from right are `1,4,12,24`. - Final output is `[24,12,8,6]`.

```
def productExceptSelf(nums):
    out, pref = [1] * len(nums), 1
    for i, x in enumerate(nums):
        out[i] = pref; pref *= x
    suff = 1
    for i in range(len(nums)-1, -1, -1):
        out[i] *= suff; suff *= nums[i]
    return out
```

Zeros: The method handles one or many zeros without branching; division would need special cases and may be forbidden. **Invariant:** Before the reverse update, `out[i]` is product strictly left; `suff` is product strictly right. **Cost:** O(n) time and O(1) auxiliary space excluding output.

34. Longest Substring Without Repeating Characters (3)

Method: Sliding window with last-seen indices; jump left past the duplicate. - For `abba`, at second `b`, move left from 0 to 2. - At second `a`, old index 0 is outside the window, so do not move left backward. - Track `left = max(left, last[ch]+1)`.

```
def lengthOfLongestSubstring(s):
    last, left, best = {}, 0, 0
    for right, ch in enumerate(s):
        if ch in last: left = max(left, last[ch] + 1)
        last[ch] = right
        best = max(best, right - left + 1)
    return best
```

Invariant: `s[left:right+1]` has unique characters after the left update. **Cost:** O(n) time, O(alphabet) space. **Extension:** At most `k` distinct characters uses counts and a shrinking while-loop instead of index jumps.

35. Minimum Window Substring (76)

Method: Expand until all required multiplicities are satisfied, then shrink while valid. - `need` stores target counts; `missing` counts required character instances, not distinct keys. - A new character reduces `missing` only if its old window count was below need. - During shrink, removing a required instance makes the window invalid and stops contraction.

```
from collections import Counter
def minWindow(s, t):
    need, missing = Counter(t), len(t)
    left = 0; best = (float("inf"), 0, 0)
    for right, ch in enumerate(s):
        if need[ch] > 0: missing -= 1
        need[ch] -= 1
        while missing == 0:
            if right-left+1 < best[0]: best = (right-left+1, left, right+1)
            old = s[left]; need[old] += 1; left += 1
            if need[old] > 0: missing += 1
    return s[best[1]:best[2]]
```

Invariant: Negative `need[ch]` means surplus copies in the current window. **Cost:** O(|s|+|t|); each pointer

advances at most |s| times.

36. Daily Temperatures (739)

Method: Monotonic decreasing stack of unresolved indices. - Each index waits for the first future warmer temperature. - On warmer input, pop while current temperature is greater and fill distances. - Remaining indices have answer zero.

```
def dailyTemperatures(t):
    ans, st = [0] * len(t), []
    for i, x in enumerate(t):
        while st and t[st[-1]] < x:
            j = st.pop(); ans[j] = i - j
        st.append(i)
    return ans
```

Invariant: Stack temperatures are nonincreasing from bottom to top; every stacked index is unresolved. **Cost:** O(n), because each index is pushed and popped at most once. **Extension:** Next greater element uses the same stack; largest rectangle changes what is stored and when area is finalized.

37. Find Median from Data Stream (295)

Method: Max heap for lower half, min heap for upper half; sizes differ by at most one. - Insert into lower by default, move its maximum to upper. - If upper becomes larger, move its minimum back. - Lower owns the extra element for odd counts.

```
import heapq
class MedianFinder:
    def __init__(self): self.lo, self.hi = [], []
    def addNum(self, x):
        heapq.heappush(self.lo, -x)
        heapq.heappush(self.hi, -heapq.heappop(self.lo))
        if len(self.hi) > len(self.lo):
            heapq.heappush(self.lo, -heapq.heappop(self.hi))
    def findMedian(self):
        if len(self.lo) > len(self.hi): return -self.lo[0]
        return (-self.lo[0] + self.hi[0]) / 2
```

Invariants: Every lower value is <= every upper value; size balance is `len(lo)  {len(hi), len(hi)+1}`. **Cost:** O(log n) insert, O(1) median, O(n) space.

38. Implement Trie (208)

Method: Each edge is a character; terminal marker distinguishes a complete word from a prefix. - Insert walks/creates nodes. - Search requires all edges and terminal marker. - Prefix check requires only all edges.

```
class Trie:
    def __init__(self): self.root = {}
    def insert(self, word):
        node = self.root
        for ch in word: node = node.setdefault(ch, {})
        node["#"] = True
    def _walk(self, text):
        node = self.root
        for ch in text:
            if ch not in node: return None
            node = node[ch]
```

```
        return node
    def search(self, word):
        node = self._walk(word)
        return node is not None and "#" in node
    def startsWith(self, prefix):
        return self._walk(prefix) is not None
```

Cost: O(length) per operation; memory proportional to distinct stored prefixes. **C3 angle:** Prefix indexes fit entity paths, feature namespaces, configuration keys, and command autocomplete.

39. Accounts Merge (721)

Method: Union accounts sharing an email, then group emails by representative. - Assign each email an owner account index. - Seeing an existing email unions current account with prior owner. - Group unique emails by final root and sort within each group.

```
def accountsMerge(accounts):
    n = len(accounts); parent = list(range(n))
    def find(x):
        while x != parent[x]:
            parent[x] = parent[parent[x]]; x = parent[x]
        return x
    def union(a, b):
        ra, rb = find(a), find(b)
        if ra != rb: parent[rb] = ra
    owner = {}
    for i, account in enumerate(accounts):
        for email in account[1:]:
            if email in owner: union(i, owner[email])
            else: owner[email] = i
    groups = {}
    for email, i in owner.items():
        groups.setdefault(find(i), []).append(email)
    return [[accounts[r][0]] + sorted(es) for r, es in groups.items()]
```

Invariant: Union-find represents transitive identity: A sharing with B and B with C merges all three. **Cost:** Near O(E (n)) plus sorting emails. Add union-by-size for stronger worst-case behavior.

40. Serialize and Deserialize Binary Tree (297)

Method: Preorder with explicit null markers; deserialize consumes the token stream recursively. - Values alone are ambiguous; nulls encode shape. - Preorder token for node is followed by complete left and right encodings. - Decoder advances exactly once per token.

```
class Codec:
    def serialize(self, root):
        out = []
        def dfs(x):
            if not x: out.append("#"); return
            out.append(str(x.val)); dfs(x.left); dfs(x.right)
        dfs(root)
        return ",".join(out)
    def deserialize(self, data):
        it = iter(data.split(","))
        def dfs():
```

```
            token = next(it)
            if token == "#": return None
            x = TreeNode(int(token))
            x.left, x.right = dfs(), dfs()
            return x
        return dfs()
```

Invariant: The serialized grammar is prefix-decodable because each non-null node demands exactly two child encodings. **Cost:** O(n) time and output; O(h) recursion stack. **Production note:** Add schema/version framing and input limits before accepting untrusted serialized trees.

Pattern decision tree

Start with constraints, then choose the row whose invariant you can state precisely. "Data structure" is not an explanation; the ownership or ordering property is.

First question	If yes	Next question	Default
Is the answer about a contiguous range?	array/string window	Are values nonnegative or is validity monotone as left moves?	sliding window / two pointers
Is the answer about a contiguous range?	array/string window	Do negatives break monotonicity, but sums matter?	prefix sum + map
Is input sorted or predicate monotone?	ordered search space	Searching a value or a minimum feasible answer?	binary search / binary search on answer
Must choices be enumerated?	search tree	Can an invalid prefix be detected early?	guarded backtracking
Is shortest unweighted distance required?	graph/grid	One source or many simultaneous sources?	BFS / multi-source BFS
Are dependencies directed?	graph	Need order or cycle detection?	topological sort / three-color DFS
Are relationships transitive and merged online?	components	Need repeated union/connectivity?	union-find
Is "next greater/smaller" involved?	ordered neighbors	Does each item resolve once?	monotonic stack
Is top/bottom `k` needed?	ranked stream	Is `k` much smaller than `n`?	size-k heap
Is median needed online?	ranked stream	Can all values be retained?	two heaps

Structural signal	Clarifying question	Preferred pattern	Common trap
linked-list offset/cycle	fixed distance or meeting point?	fast/slow pointers	losing head without dummy
nested delimiters/path	does latest unmatched item decide?	stack	regex or repeated replacement
intervals	union, peak overlap, or assignment?	sort+merge / event sweep / heap	endpoint semantics unstated
tree hierarchy	top-down state or bottom-up evidence?	DFS with explicit return contract	global mutable state
cache with recency	are get and put strict O(1)?	map + doubly linked list	ordered array updates
repeated time-key lookup	append timestamps ordered?	per-key list + binary search	scanning history
exact ownership under races	can one conditional write decide?	unique constraint / CAS	cache-based lock as truth
expensive repeated subproblems	state dimensions small and bounded?	memoization or bottom-up DP	greedy without exchange proof

Sliding-window diagnostic	Answer	Action
When right expands, can validity only get worse?	yes	shrink left while invalid
When left advances, can validity only improve?	yes	ordinary sliding window works
Do negative numbers make the sum rise or fall unpredictably?	yes	abandon sum-based sliding window
Need exact multiplicities of target symbols?	yes	counts plus `missing` or `formed`
Need unique symbols only?	yes	last-seen jump or frequency set

Binary-search diagnostic	Contract
Find exact target	discard half that provably cannot contain target
Find first true	`hi` remains feasible; set `hi=mid`
Find last true	bias midpoint upward; set `lo=mid`
Search rotated array	identify a sorted half before testing membership
Search answer	define monotone `feasible(x)` and prove bounds

Complexity cheat sheet

Let `n` be items, `V/E` graph vertices/edges, `R/C` grid dimensions, `k` retained rank count, `A` numeric amount, and `L` candidate length.

Pattern / operation	Time	Auxiliary space	Interview caveat
hash lookup / update	average O(1)	O(n) table	worst case depends on hashing
sort then scan	O(n log n)	language-dependent	Python sort uses O(n) worst-case temp
two pointers	O(n)	O(1)	prove discarded side is dominated
sliding window	O(n)	O(alphabet)	each pointer advances at most n
prefix sum + frequencies	O(n)	O(n)	seed identity prefix
binary search	O(log n)	O(1)	bounds and duplicate policy matter
binary search on answer	O(cost(feasible) · log range)	predicate-dependent	range is values, not item count
heap top-k	O(n log k)	O(k)	full sorting may be simpler
BFS / DFS graph	O(V+E)	O(V)	mark visited at enqueue/discovery
grid traversal	O(RC)	O(RC) worst case	recursion may overflow
union-find sequence	O(n (n)) amortized	O(n)	use compression and union-by-size
monotonic stack	O(n)	O(n)	amortized: each item pops once
trie operation	O(L)	O(total distinct prefixes)	alphabet representation affects constants
1-D amount DP	O(nA)	O(A)	pseudo-polynomial in numeric A
subset backtracking	O(2^n · n) output-sensitive	O(n) stack	pruning improves practical, not worst case

Pattern / operation	Time	Auxiliary space	Interview caveat
permutation backtracking	O(n! · n)	O(n) stack	output alone has factorial size
LRU get/put	O(1)	O(capacity)	needs map plus constant-time order edits
two-heap median insert/query	O(log n) / O(1)	O(n)	state both ordering and balance invariants

Python operation reminders | Operation | Cost | Note | |—|—:|—| | list append/pop end | amortized O(1) | front pop is O(n) | | `deque` append/popleft | O(1) | use for BFS | | dict/set lookup | average O(1) | keys must be hashable | | heap push/pop | O(log n) | root peek O(1) | | string concatenation in loop | can be $O(n^2)$ | collect list then `"".join` | | slicing `a[l:r]` | O(r-l) | hidden copies can alter space claims | | `sorted` | O(n log n) | key computed once per item | | recursion | O(depth) frames | Python depth is limited |

Interview talk scripts

Opening, 25 seconds: "I'll restate the input and output, clarify empty input, duplicates, ordering, mutation, and size limits, then give a simple baseline. I'll optimize only after naming the property that removes repeated work. I'll trace one normal and one boundary case before coding."

From brute force to pattern: "The direct solution checks every candidate range, so it repeats work across overlapping ranges and costs $O(n^2)$. Because the requirement is monotone as the window moves, I can retain a valid window and move each pointer at most once, reducing this to O(n). The invariant is that the current window satisfies _____ after the shrink loop."

Prefix sums: "A range sum `i..j` equals `prefix[j]-prefix[i-1]`. At the current prefix `p`, every earlier prefix `p-k` creates a valid range, so I store frequencies, not merely membership. I seed zero once to count ranges beginning at index zero."

Binary search on answer: "I'm not searching the input; I'm searching candidate answers. `feasible(x)` is monotone because increasing x can never make ___ worse. My bounds contain the answer, `hi` remains feasible, and the loop returns the first true value."

BFS: "All edges have equal cost, so queue order is nondecreasing distance. I mark a node when enqueued, not when dequeued, preventing duplicate frontier entries. With multiple simultaneous sources, I seed all of them at distance zero."

Backtracking: "The state records only the choices needed to validate a prefix. I choose, recurse, and undo symmetrically. The guards ensure invalid prefixes never enter the tree, and at a leaf I copy the current path because it will be mutated."

Dynamic programming: "Let `dp[x]` mean ***. The recurrence uses only strictly smaller solved states, the base case is*** , and iteration order guarantees dependencies are ready. I'll use rolling storage because only the previous row is needed."

Concurrency follow-up: "If two callers can claim the same object, an in-memory check is insufficient. I need one atomic conditional write or uniqueness constraint as the linearization point; retries carry an idempotency key and return the prior result."

Testing while coding: "I'll test empty/minimal input, a typical case, duplicates, an impossible case, and the boundary that changes pointer or inequality behavior. Then I'll state time and auxiliary space, including recursion and output."

When stuck: "I can preserve correctness with the $O(n^2)$ version first. The repeated dimension is _____. If I summarize that history with a map/heap/prefix state, each new element can be processed once."

Rapid variants and follow-ups

Base problem	Interviewer twist	Adaptation
Koko	maximize minimum allocation	last-true binary search
Unique Paths	blocked cells	set blocked cell DP to zero
Coin Change	count combinations	coins outer loop, amounts increasing
Coin Change	count permutations	amounts outer loop, coins inner
Valid Parentheses	ignore quoted text	parser state for quote/escape
Rotting Oranges	weighted spread times	multi-source Dijkstra
Merge Intervals	return maximum overlap	sorted events or min heap
LRU	frequency before recency	LFU: frequency buckets + DLLs
Subarray Sum	longest range	earliest prefix index, not frequency
Generate Parentheses	multiple bracket types	remaining counts plus expected-close stack
Rotated Search	duplicates	shrink equal ambiguous ends; worst O(n)
Word Search	many dictionary words	trie-guided DFS
Hit Counter	distributed	time buckets + associative merge
Continuous Subarray	longest divisible range	earliest remainder index and maximize gap
Top K Words	streaming	counts plus heap; define update policy
Course Schedule	return cycle	DFS parent chain or residual graph
Islands	dynamic additions	union-find land cells as activated
Median Stream	sliding window	delayed-deletion heaps
TimeMap	out-of-order writes	ordered structure or sort/compact
Trie	delete	reference counts and safe node pruning

C3-oriented mini-drills

Prompt	First sentence	Core structure	Failure to mention
deduplicate model events	“Identity and time window define duplicate.”	event-ID map with expiry	unbounded memory
latest config as of time	“This is predecessor lookup per key.”	sorted versions + binary search	out-of-order updates
execute feature DAG	“Dependencies require topological order.”	indegrees + queue	cycle reporting
group connected entities	“Shared identifiers create transitive components.”	union-find	conflicting canonical names
rolling metric median	“Median needs balanced lower/upper halves.”	two heaps	deletion/window semantics
limit requests per tenant	“Window semantics and tolerated burst come first.”	deque/buckets/token bucket	distributed overshoot
merge alert windows	“Sort by start and preserve disjoint output.”	interval sweep	touching endpoint policy
nearest healthy service	“Equal-cost hops imply BFS.”	queue + visited	stale health snapshot
schedule constrained jobs	“I’ll separate dependency order from resource slots.”	topo order + heap	starvation
cache expensive inference	“Key must include every result-affecting input.”	hash + LRU/TTL	tenant/ACL leakage

Final five-minute self-check

- Can I state the invariant before naming the container?
- Did I ask whether duplicates, mutation, and ordering guarantees exist?
- Did I choose the correct endpoint inequality?

- Does every loop make progress on every branch?
- Did I seed the identity state: zero prefix, root, source frontier, or DP base?
- Do I mark visited at discovery rather than completion?
- In backtracking, is every mutation undone?
- In recursion, is the return contract one sentence?
- Under retries, can the same logical operation happen twice?
- Is stated auxiliary space honest about stack, slicing, maps, and output?
- Can I explain why the discarded candidate cannot be optimal?
- Can I produce one adversarial input that breaks the tempting alternate?

Boundary-case clinic

These are the short tests to speak before running code. For each row, predict the failure mode before looking at the correction.

Pattern	Adversarial input	Tempting bug	Correction
recurring decimal	`-1/-2`, `1/2`, `1/6`	sign/terminating/cycle confusion	sign first; remainder map
k-diff	duplicates with `k=0`	count index pairs	count keys with frequency >1
binary answer	answer equals lower bound	skip boundary	inclusive proven bounds
rolling DP	one row or column	uninitialized edge	identity row of ones
coin change	no coin 1	return sentinel	convert unreachable to -1
parentheses	starts with closer	pop empty stack	check empty before pop
two pointers	empty/one element	invalid right index	define minimal-input policy
grid BFS	no fresh cells	return wrong minute	initialize time/result carefully
intervals	`[1,2]`, `[2,3]`	wrong overlap policy	clarify closed vs half-open
LRU	capacity one	broken endpoints	sentinels and update-before-evict
prefix counts	range starts at zero	miss it	seed `{0:1}`
backtracking	append mutable path	all outputs identical	append a copy
rotated search	equal endpoints	cannot choose sorted half	shrink ambiguity if duplicates allowed
word search	same cell reused	false positive	mark during current path
hit counter	hit exactly 300 sec old	off-by-one	define `[t-299,t]` window
linked list	delete head	no predecessor	dummy head
sliding unique	`abba`	move left backward	`max(left,last+1)`
minimum window	repeated target chars	track distinct only incorrectly	multiplicities / missing instances
monotonic stack	equal temperatures	treat equal as warmer	strict comparison
topological sort	disconnected DAG	start from one node	seed all zero-indegree nodes
union-find	long chain	quadratic finds	compression + union by size
serialization	sparse shape	values cannot reconstruct shape	explicit null markers

Proof templates

Discard proof — two pointers: "Assume the left boundary is no taller than the right. Any pair using this left boundary at a smaller width has limiting height no greater than the current left height, so it cannot beat the current pair. Therefore advancing left discards no better answer."

Greedy reset proof — gas station: "If the accumulated gain from candidate start `s` through `i` is negative, then any start between `s` and `i` reaches `i` with no more fuel after removing a nonnegative/less-negative prefix; none can succeed. The next candidate is `i+1`."

Prefix-map proof: "For current prefix `P[j]`, a prior boundary `i` forms target `k` exactly when `P[i]=P[j]-k`.

Counting all prior occurrences counts each valid range once by its right endpoint."

BFS shortest-path proof: "The FIFO queue processes nodes in nondecreasing edge count. The first discovery of a node therefore uses a shortest path; marking then prevents longer duplicate paths."

Topological-sort proof: "Only zero-indegree nodes have all prerequisites satisfied. Removing one and its outgoing edges preserves that condition. If nodes remain with no zero-indegree candidate, the residual directed graph contains a cycle."

Dynamic-programming proof: "The recurrence enumerates every possible final choice and combines it with an optimal solution to the remaining smaller state. Taking the best over those choices is both feasible and no worse than any complete solution."

Code-review checklist

Area	Questions to ask while scanning
contract	Are null, empty, impossible, duplicate, and mutation behaviors defined?
indices	Is each bound inclusive or exclusive, and can it cross safely?
maps	Is lookup done before update when current item must not match itself?
queues	Is visited marked on enqueue and is level size snapshotted?
heaps	Is sign inversion/tie-breaking correct and is heap size bounded?
recursion	Is there a base case before dereference, and is depth safe?
backtracking	Does every choose have an undo on every return path?
arithmetic	Can integer overflow occur in other languages; are floats avoidable?
sorting	Does mutation matter; is comparator transitive and tie policy exact?
complexity	Are slicing, sorting, recursion stack, output, and hash storage counted?

Ten-minute mixed mock

1. **Minute 0–1:** Explain why sliding window fails for exact sum with negatives; derive prefix counts.
2. **Minute 1–2:** Trace first-true binary search and state why `hi=mid`, not `mid-1`.
3. **Minute 2–3:** Write a BFS skeleton with visited-on-enqueue and level separation.
4. **Minute 3–4:** Convert recursive tree evidence into a one-sentence return contract.
5. **Minute 4–5:** Explain one monotonic-stack amortized O(n) proof.
6. **Minute 5–6:** Compare heap top-k with full sort and bucket counting.
7. **Minute 6–7:** State LRU's two invariants and four pointer edits.
8. **Minute 7–8:** Derive a DP state, recurrence, base, order, and answer location.
9. **Minute 8–9:** Name five boundary tests without executing code.
10. **Minute 9–10:** Give the final complexity and one production concurrency follow-up.

Last-word vocabulary

Say this	It signals
"linearization point"	you know where a concurrent effect becomes singular
"output-sensitive"	exponential output may make exponential work unavoidable
"amortized"	a costly operation is bounded across the full sequence
"half-open interval"	endpoint behavior is deliberate
"stable idempotency key"	retries preserve logical identity

Say this	It signals
“mark on discovery”	graph traversal avoids duplicate frontier work
“monotone predicate”	binary search on answer is justified
“exchange argument”	greedy choice has a proof
“state compression”	DP memory reduction preserves needed dependencies
“source of truth”	caches accelerate but do not grant ownership

Python-to-Java translation traps

Python shorthand	Java interview equivalent	Trap
`dict.get(k,0)`	`map.getOrDefault(k,0)`	update after counting current item
`defaultdict(list)`	`computeIfAbsent`	avoid sharing one mutable default
`deque.popleft()`	`ArrayDeque.removeFirst()`	never use `ArrayList.remove(0)` for BFS
`heapq` min heap	`PriorityQueue`	comparator overflow from subtraction
negative-value max heap	reverse comparator	define deterministic tie order
`s[l:r]`	`substring(l,r)`	modern Java copies; still count output
tuple key	small record/class	implement stable equality and hash
recursion	recursive helper	stack overflow on deep grids/trees
arbitrary integers	`long` where products/sums grow	cast before multiplication
list sort with key	comparator	comparator must be transitive

Portable coding rules - Use half-open ranges `[left,right)` when designing helpers. - Promote to 64-bit before multiplying width, rate, timestamps, or counts. - Avoid comparator subtraction; use language comparison helpers. - Use an explicit small state object when tuple meaning becomes unclear. - Prefer iterative graph traversal when depth may approach input size. - State whether library ordered maps/caches are allowed before relying on them. - Keep algorithm and I/O parsing separate so the core can be tested directly. - In Java, define `equals`/`hashCode` consistently for map/set identity. - In Python, do not use mutable default arguments for accumulators. - In either language, choose descriptive boundary names over `i/j` in complex sweeps.

Final recall grid

If you hear…	Say immediately
recurring decimal	remainder → output index
minimum feasible rate	binary search the answer
exact subarray sum	prefix counts, seed zero
divisible subarray	earliest prefix remainder
simultaneous spread	multi-source BFS
last-used eviction	map plus doubly linked order
circular feasibility	total check plus greedy reset
path combinations	guarded backtracking
delete from list end	dummy plus fixed pointer gap